



Wrocław University of Technology

Dockers

Tomasz Walkowiak

Why containers?

Shipping code to the server is a hard job

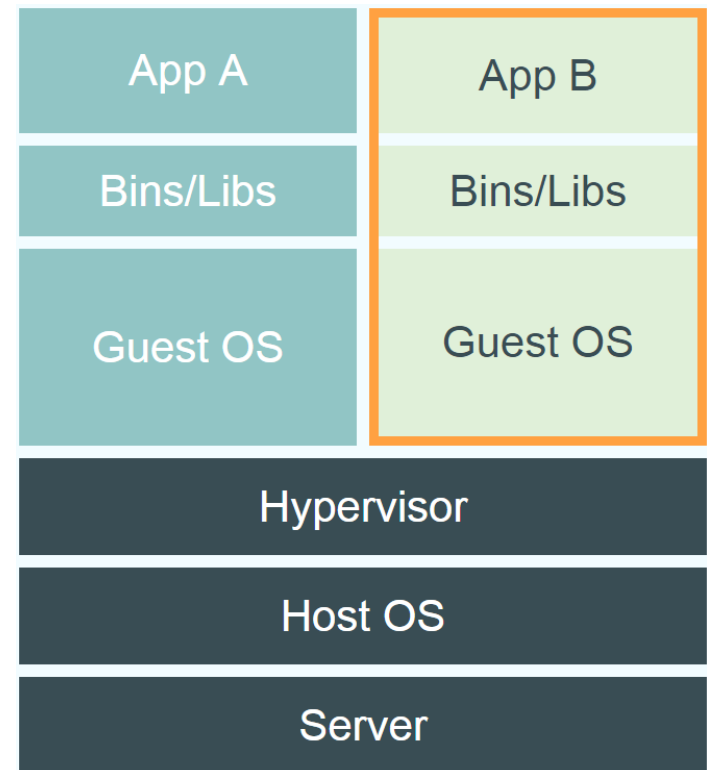
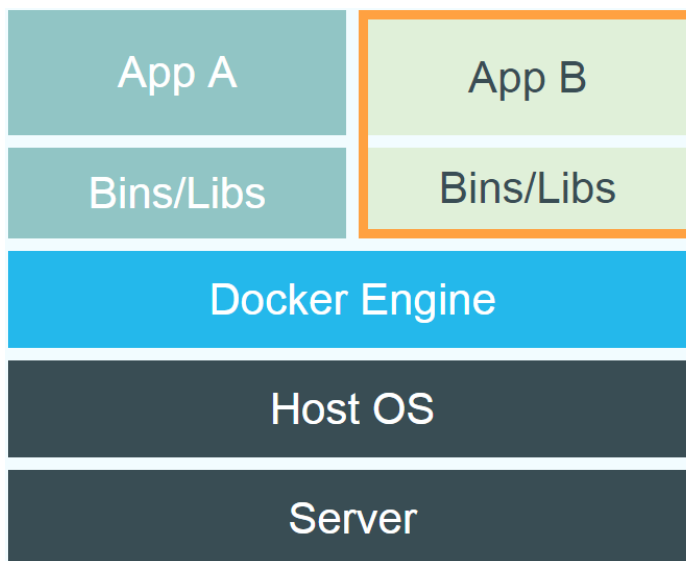


Dependencies, dependencies,....



Container/Docker

- Linux containers (LXC) are “lightweight” VMs
- Docker is a commoditized LXC technique that dramatically simplifies the use of LXC





Container architecture

- Linux Kernel

- namespaces: isolation. Much like FreeBSD Jails, Solaris Zones. Started in 2002.

ns:mnt,pid,net,ipc,uuts,user,proc into
/proc/pid/ns

- cgroup (control group): resource management, resource accounting/tracking solution. Started in 2006 (Google). Merged into kernel 2008 (2.6.24)

The combination of
NAMESPACE and
CGROUP enables
container technology!!



LXC technique

- Linux kernel provides the “control groups” (cgroups) functionality
 - allows limitation and prioritization of resources (CPU, memory, block I/O, network, etc.) without the need for starting any VM
- “namespace isolation” functionality
 - allows complete isolation of an applications' view of the operating environment, including process trees, networking, user IDs and mounted file systems.



Unique features

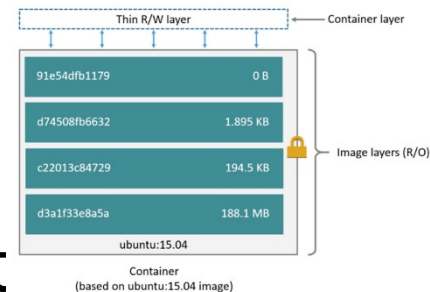
- Containers running in the user space
- Each container has
 - Own process space
 - Own network interface
 - Own /sbin/init (coordinates the rest of the boot process and configures the environment for the user)
 - Run stuff as root
- Share kernel with the host
- No device emulation

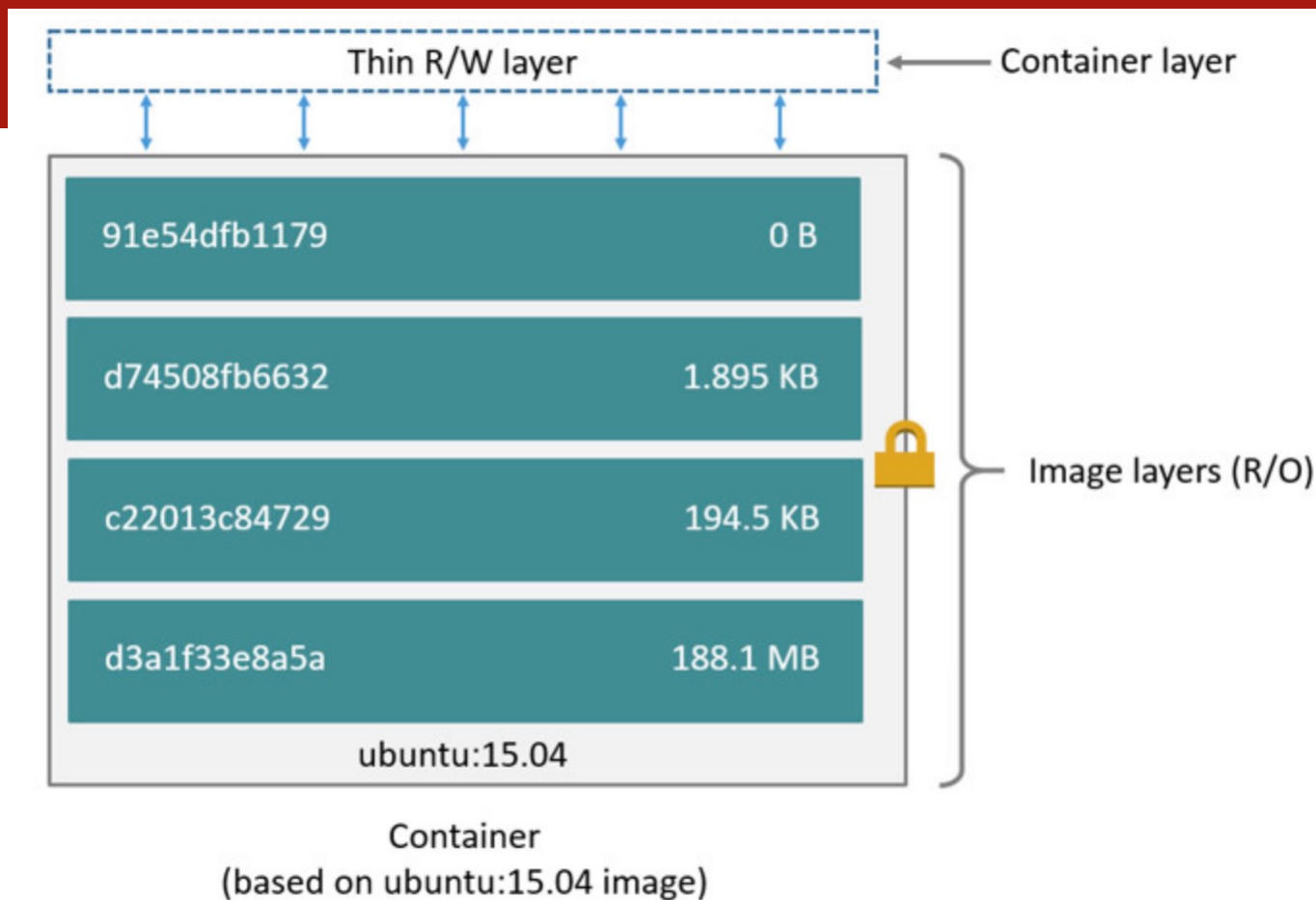
Almost no overhead

- processes are isolated, but run straight on the host
- CPU performance = native performance
- memory performance = a few % shaved off for (optional) accounting
- network performance = small overhead; can be reduced to zero

And CoW!

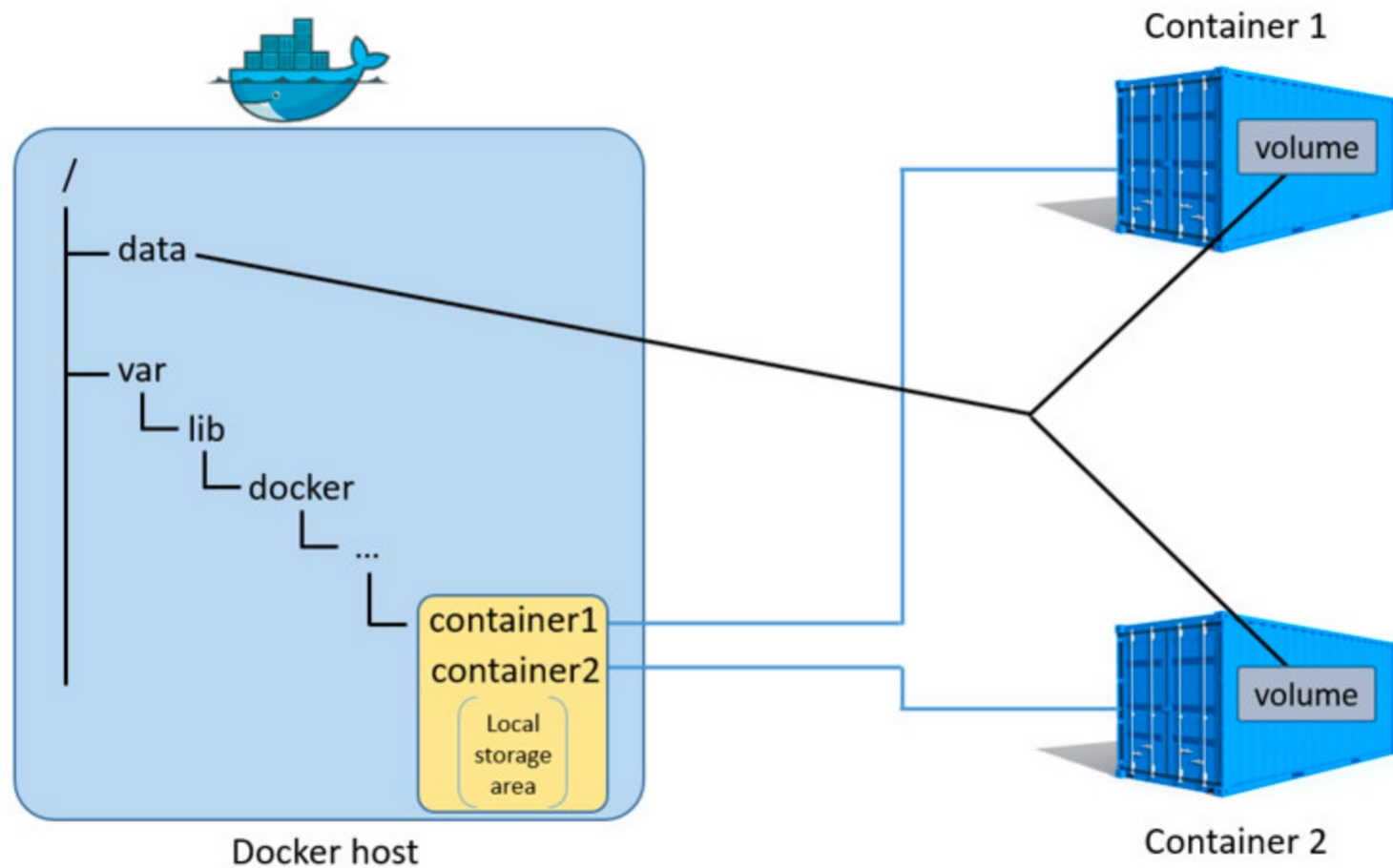
- Filesystem Property
 - CoW: Copy On Write.
 - When container starts, image (storage,libs,apps,etc) is available. Just mount. This is fast!
 - Storage keeps track of what has
 - AUFS,overlay (file level) look into /var/lib/docker/....
 - DeviceMapper (RHEL) (block level)
 - BTRFS, ZFS (FS level)







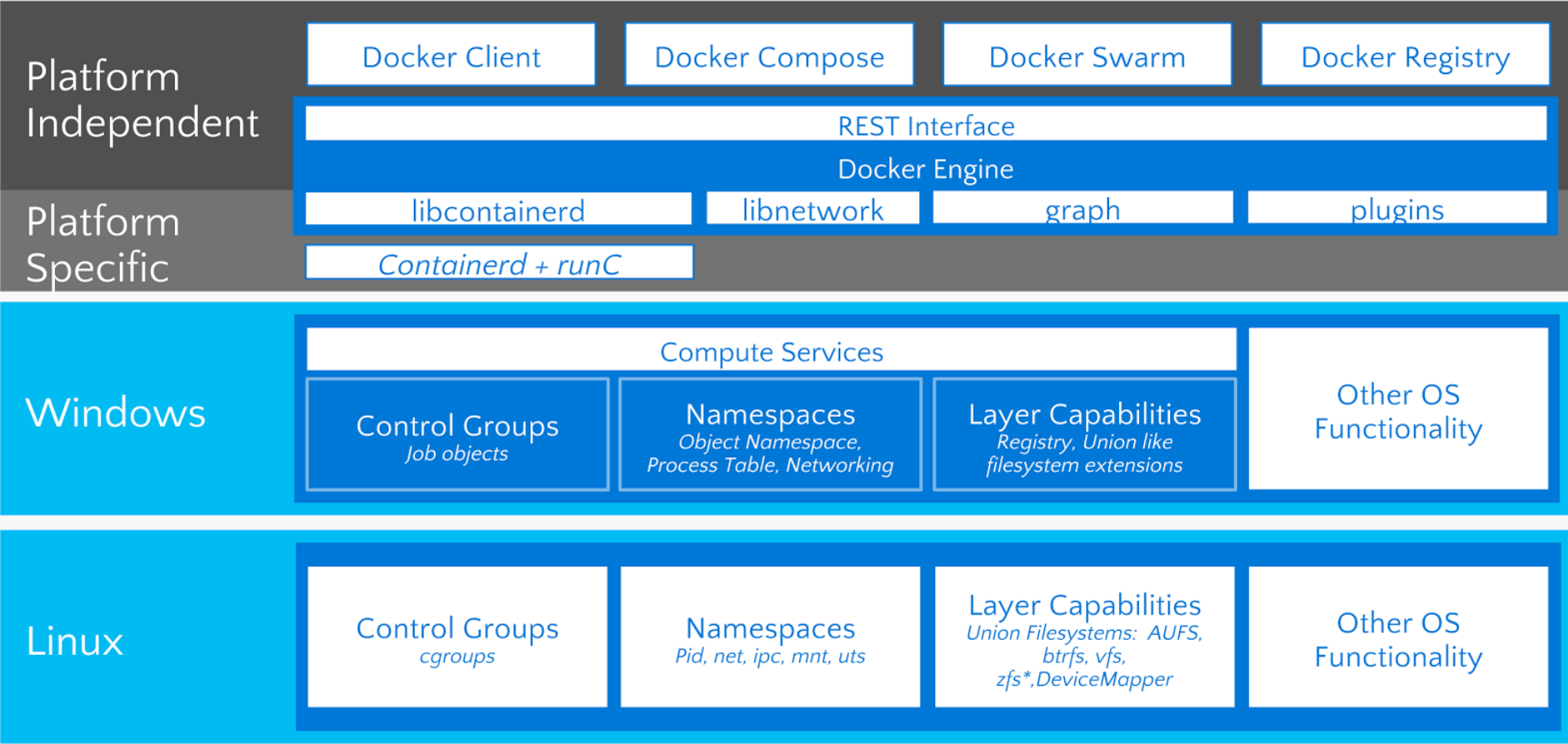
The combination of
NAMESPACE, CGROUP,
CoW enables container-
technology!!





What is docker

- Open Source engine to commoditize LXC
- using copy-on-write for quick provisioning
- allowing to **create and share *images***
- **standard format** for containers
- standard, *reproducible* way to *easily* build *trusted* images (Dockerfile, Stackbrew...)



Docker on Windows

Two types of containers:

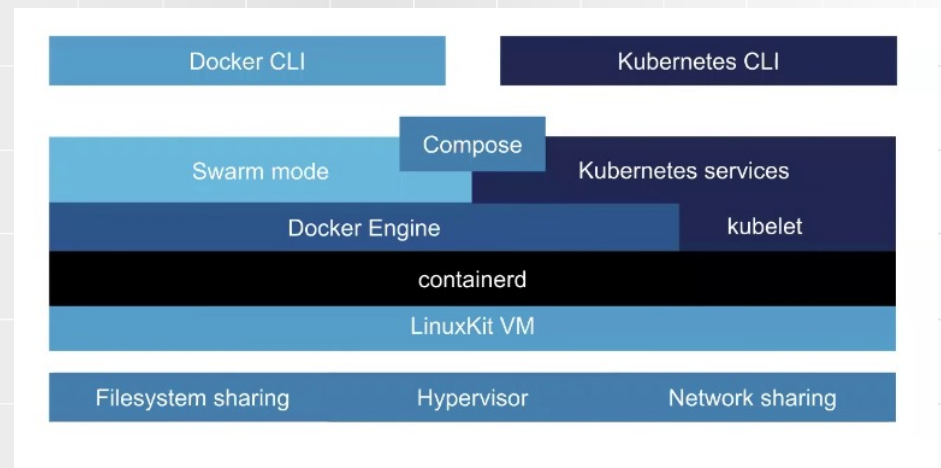
- Windows containers (only on some distributions) :
 - Runs on Windows Kernel
 - need to match the version of the OS where the container is running
- Linux containers:
 - In the first version VM was Hyper-V VM
 - runs on Linux virtual machine
 - Hyper-V
 - VHD to store container images and configs
 - In new architecture uses WSL 2
 - runs on LinuxKit container
 - bootstrapping distro
 - **data store distro**

Docker on Windows continued

- Windows Subsystem for Linux (WSL) 2
 - full Linux kernel built by Microsoft
 - allows Linux distributions to run without managing VM's
- Main change:
 - don't run the LinuxKit distro in a VM but in a container
- WSL 2 new features:
 - Kubernetes support
 - support for latest stable Docker Daemon
 - VPN-friendly networking

Docker on MacOS

- Docker needs a **LinuxKit VM** to run on Mac
- Uses HyperKit virtualization
- Has its own **osxfs** file sharing solution
- Multi-Architectural



<https://collabnix.com/how-docker-for-mac-works-under-the-hood/>

Docker on ARM Processors

- Normally docker tries to use arm images
- If arm version is not available uses x86
- **Rosetta2** emulator to create bridge between Intel and Apple Silicon
- To force image build for x86 **--platform=linux/amd64**
- Some libraries hasn't distribution for arm (build image for x86)

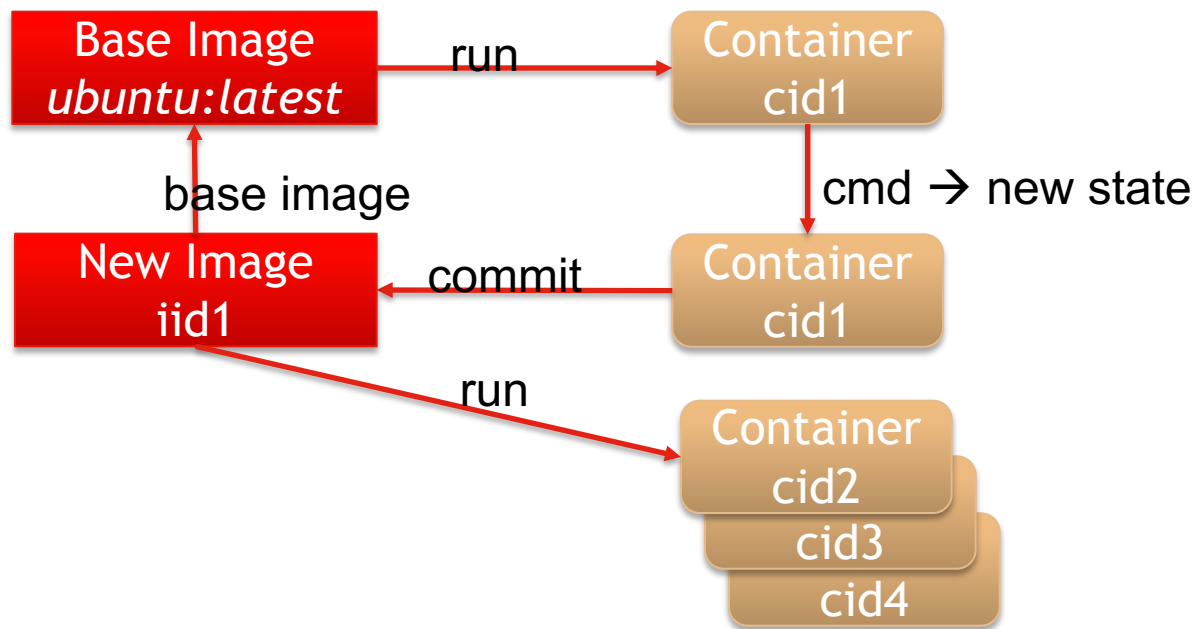
Sources (MacOS)

- <https://www.docker.com/blog/the-magic-behind-the-scenes-of-docker-desktop/>
- <https://www.docker.com/blog/apple-silicon-m1-chips-and-docker/>
- <https://collabnix.com/how-docker-for-mac-works-under-the-hood/>
- <https://www.computerworld.com/article/3597949/everything-you-need-to-know-about-rosetta-2-on-apple-silicon-macs.html>



Building docker image

- With run/commit commands
 - 1) `docker run ubuntu bash`
 - 2) `apt-get install this and that`
 - 3) `docker commit <containerid> <imagename>`
 - 4) `docker run <imagename> bash`
 - 5) `git clone git://.../mycode`
 - 6) `pip install -r requirements.txt`
 - 7) `docker commit <containerid> <imagename>`
 - 8) repeat steps 4-7 as necessary
 - 9) `docker tag <imagename> <user/image>`





Authoring image with a dockerfile

- A sample dockerfile

```
FROM ubuntu:16.04
```

```
RUN apt-get -y update && \  
  apt-get install -y apt-utils iputils-ping iputils-tracepath cmake build-essential
```

```
RUN cd /tmp/apache-couchdb-* && ./configure && make install
```

```
EXPOSE 8101
```

```
CMD ["/usr/local/bin/couchdb"]
```

Run the command to build:

```
docker build -t your_account/couchdb .
```



Real example

FROM clarinpl/python:3.6

WORKDIR /home/worker

COPY ./src ./src

COPY ./main.py .

COPY ./requirements.txt .

RUN python3.6 -m pip install -r
requirements.txt

CMD ["python3.6", "main.py", "service"]

FROM ubuntu:16.04

RUN apt update && \

apt install -y software-properties-common && \

add-apt-repository ppa:deadsnakes/ppa && \

apt update && \

apt install -y python3.6 \

apt-transport-https python3.6-dev subversion git wget

RUN wget -q -O - http://apt.clarin-pl.eu/KEY.gpg | apt-key add - && \

echo 'deb https://apt.clarin-pl.eu/ /' > /etc/apt/sources.list.d/clarin.list &

apt-get update

RUN update-alternatives --install \

/usr/bin/python python /usr/bin/python3.6 10 && \

wget https://bootstrap.pypa.io/get-pip.py && \

python get-pip.py && \

rm get-pip.py && \

pip install --upgrade pip

ENV PIP_EXTRA_INDEX_URL=https://pypi.clarin-pl.eu

ENV LANG C.UTF-8

ENV LC_ALL C.UTF-8

docker build -t clarinpl/python:3.6 .



Docker Hub

- Public repository of Docker images
 - <https://hub.docker.com/>
 - docker search [term]
- Automated: Has been automatically built from Dockerfile
 - Source for build is available on GitHub

docker login

docker push clarinpl/python:3.6



Java (JDK)

FROM ubuntu:16.04

```
RUN apt-get update && \  
    apt-get -y install \  
        openjdk-8-jdk \  
        git \  
        maven \  
        subversion
```

```
ENV JAVA_HOME /usr/lib/jvm/java-8-openjdk-amd64  
ENV PATH $JAVA_HOME/bin:$PATH
```



Dockerfile

```
FROM clarinpl/openjdk:8 as builder
```

```
WORKDIR /home/install
```

```
COPY ./module ./module
```

```
RUN cd module && \
```

```
    mvn clean && \
```

```
    mvn install
```

```
FROM clarinpl/openjdk-jre:8
```

```
WORKDIR /home/worker
```

```
COPY --from=builder /home/install/module/target/nlp.worker.any2txt.jar .
```

```
CMD ["java", "-jar", "nlp.worker.any2txt.jar"]
```

Docker-compose

- Compose is a tool for defining and running multi-container Docker applications.
- With Compose, you use a YAML file to configure your application's services.
- Then, with a single command, you create and start all the services from your configuration:
- `docker-compose up`



Docker-compose

version: '3'

services:

any2txt:

container_name: clarin_any2txt

build: ./

volumes:

- /samba:/samba
- ./config.ini:/home/worker/config.ini

working_dir: /home/worker/

entrypoint:

- java
- -jar
- nlp.worker.any2txt.jar

restart: always



Docker-compose

```
version: '3'
services:
  onlyoffice-mysql-server:
    container_name: onlyoffice-mysql-server
    image: mysql:5.7.21
    environment:
      - MYSQL_ROOT_PASSWORD=costam
    networks:
      - onlyoffice
    volumes:
      - ./config/mysql/conf.d:/etc/mysql/conf.d
      - mysql_data:/var/lib/mysql
  onlyoffice-community-server:
    container_name: onlyoffice-community-server
    image: onlyoffice/communityserver:latest
    depends_on:
      - onlyoffice-mysql-server
```



Root

- By default the container is run as root
- Solutions:
 - rootless mode
 - <https://docs.docker.com/engine/security/rootless/>
 - remapping and subordinate user and group IDs
 - <https://docs.docker.com/engine/security/rootless/>
 - Enroot (nvidia)

GPU in docker

- GPUs are important in ML
- It is possible and now element of docker
- But you need to have NVIDIA drivers installed on host
- Next:
 - install the drivers within your image
 - problems with versions
 - instruct Docker to add GPU devices to your containers at runtime.

My Code

TensorFlow/PyTorch/<FW>
+
Dependencies

Python

CPU ML libraries

AI accelerator ML libraries

AI accelerator
drivers

OS Kernel

Host OS

CPU

AI Accelerator

Your code + frameworks +
libraries

Hardware specific libraries
and hardware drivers

Heterogenous hardware
(CPU + AI accelerator)



GPU in docker

- `docker run -it --gpus all nvidia/cuda:11.4.0-base-ubuntu20.04 nvidia-smi`
- Docker compose:
- **services:**
 - test:**
 - image:** `nvidia/cuda:10.2-base`
 - command:** `nvidia-smi`
 - deploy:**
 - resources:**
 - reservations:**
 - devices:**
 - **driver:** `nvidia`
 - count:** `1`
 - capabilities:** `[gpu]`

device_ids: `['0', '3']`

Images

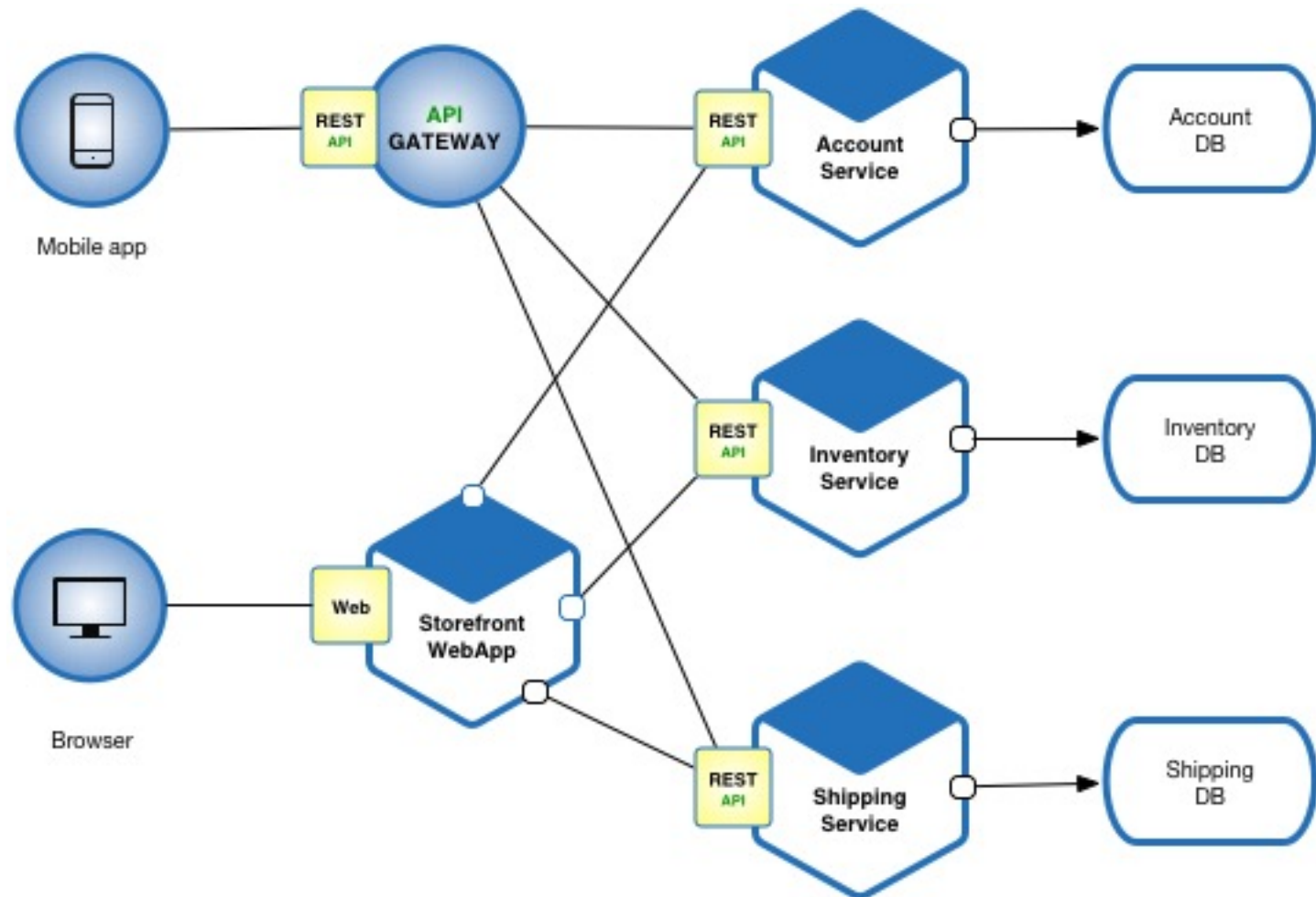
- For transformers:
 - FROM `nvidia/cuda:11.0.3-cudnn8-runtime-ubuntu20.04`
- **Pre-installed images:**
 - FROM `nvcr.io/nvidia/pytorch:22.12-py3`
- Problems:
 - Versions
 - size



My reco

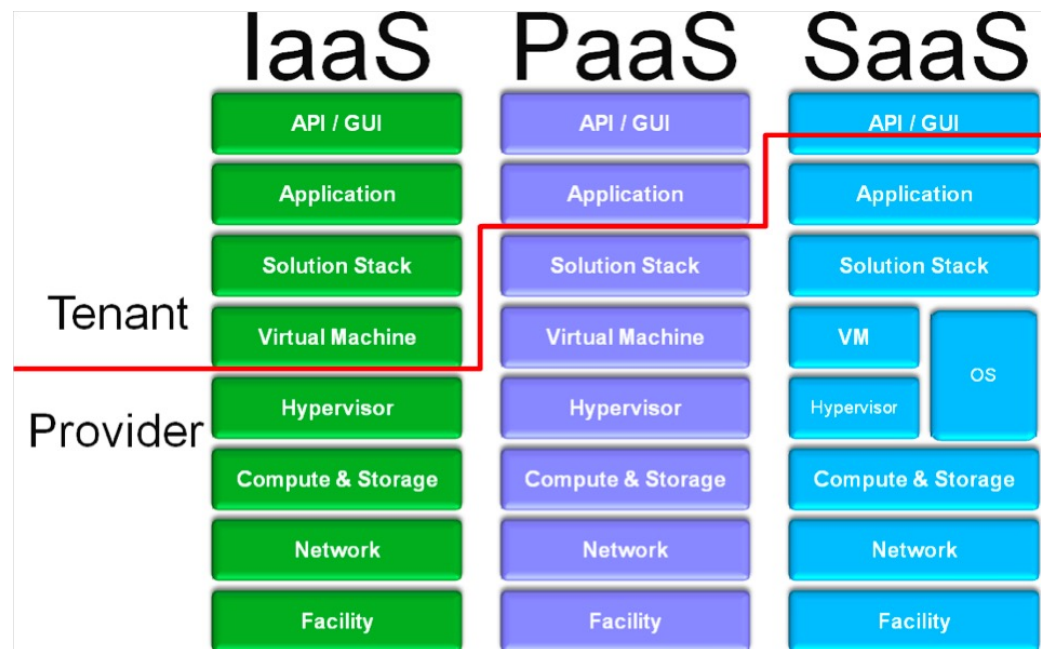
- use dockers/docker-compose
- use layers carefully
 - run , && / , group
- share images - like derivation in OOP
- make docker-images as small as possible
 - multi from
 - don't include data/models
 - mount tchem
 - be carefull with code version (latest)
- test

Microservices



Issues of cloud provider

Resource pooling, Isolation and multitenancy layer.





Docker use cases

- Development Environment
- Environments for Integration Tests
- Quick evaluation of software
- Microservices
- Multi-Tenancy
- Unified execution environment (dev → test → prod (local, VM, cloud, ...))

Not only Docker

- Podman
 - daemonless architecture
 - containers are executed under the user's privileges

- singularity

